Dynamic Programming

Dynamic Programming is a problem solving technique like divide and conquer which solves problems by dividing them into subproblems.

Divide and Conquer algorithm partition the problem into independent subproblems, solve the subproblems recursively, and then combine their solutions to solve the original problem.

While dynamic programming is applicable when subproblems are not independent.

That is, when subproblems share sub-subproblems.

Divide and Conquer may do more work than necessary, because it solves the same subproblems multiple times.

While dynamic programming solves each subproblem just once and stores the result in a table so that, it can be rapidly retrieved if needed again.

It is mainly used in optimization problems. A problem with many possible solutions for which we want to find an optimal or the best solution.

Dynamic programming works when a problem has the following characteristics

① optimal substructure

If an optimal solution

contains optimal subsolutions, then a problem exhibits optimal substructure.

② Overlapping subproblems

When a recursive algorithm would visit the same subproblems repeatedly, then a problem has overlapping subproblems.

The development of a dynamic programming algorithm can be broken into a sequence of 4 steps

① characterize the structure of an optimal solution
② Recursively define the value of an optimal solution
③ Compute the value of an optimal solution
④ Construct an optimal solution

Control Abstraction for Dynamic Programming

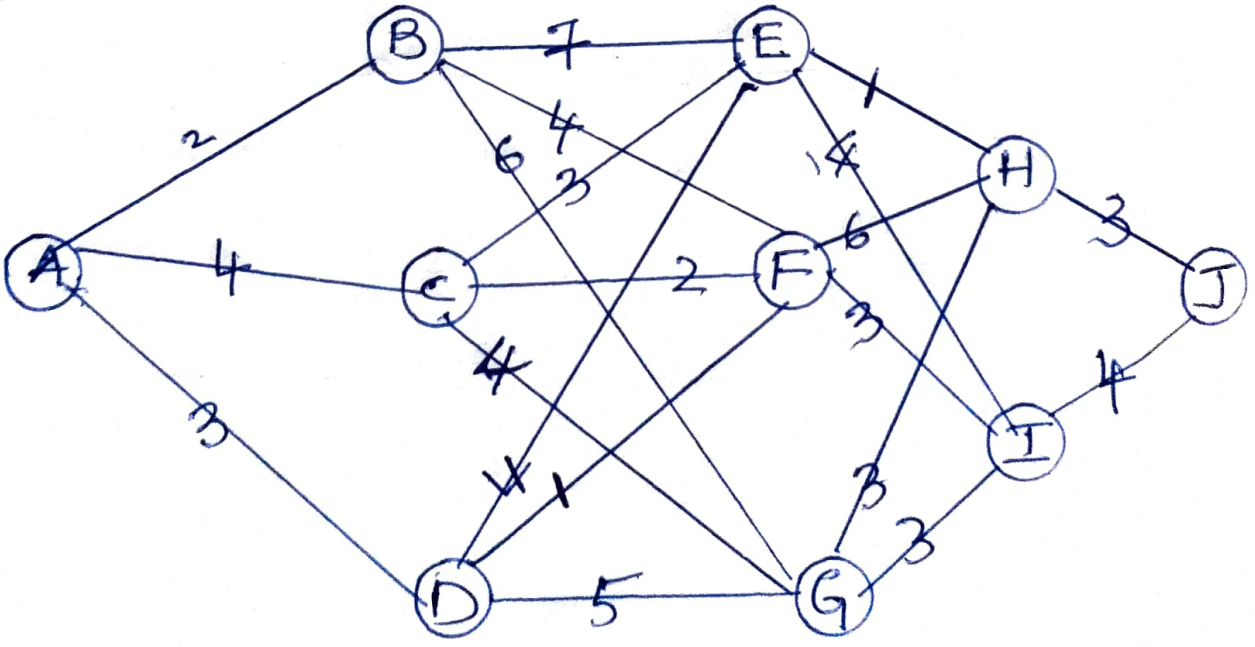
```
Algo DP(P)
if Small(P)
    if any dependent(P)
        retrieve S(P).
    else
        return S(P).
else
    divide P into smaller instances
        of P1, P2 ... Pk.
    apply DP to each of these
        subproblem
    return Combine [DP(P1) .. DP(Pk)]
```

Principle of Optimality

A Problem is said to satisfy the Principle of optimality if the subsolutions of an optimal solution of the problem are themselves optimal solutions for their subproblems.

i.e., ⇒ Optimal solution of the given problem can be obtained by using optimal solutions of its Subproblems.

Eg:- Principle of optimality is Dynamic Programming.



Let's find out shortest path from A to J using optimality principle.

For that we start from the destination J.

J

F(H) = 3

F(I) = 4.

H

F(E) = 3+1 ⇒ 4

F(F) = 3+6 ⇒ 9

F(G) = 3+3 = 6

I

F(E) = 4+4 = 8

F(F) = 4+3 = 7

F(G) = 4+3 ⇒ 7.

From these two paths [① ⇒ H as intermediate node, ② ⇒ I as intermediate node] one with shortest distance is selected for the next level.

E

F(B) = 4+7 ⇒ 11

F(C) = 4+3 ⇒ 7

F(D) = 4+4 ⇒ 8

F

F(B) = 7+4 ⇒ 11

F(C) = 7+2 = 9

F(D) = 7+1 = 8

G

F(B) = 6+6 ⇒ 12

F(C) = 6+4 ⇒ 10

F(D) = 6+5 ⇒ 11

From these values shortest values (optimal values) are selected.

B

A ⇒ 11 + 2 ⇒ 13

C

A ⇒ 7 + 4 ⇒ 11

D

A ⇒ 8 + 3 ⇒ 11.

So there is 3 shortest path from A to J with distance "11".

1 ⇒

A → C → E → H → J

4 + 3 + 1 + 3 ⇒ 11

2 ⇒

A → D → E → H → J

3 + 4 + 1 + 3 ⇒ 11

3 ⇒ A → D → F → I → J

3 + 1 + 3 + 4 ⇒ 11

Here we reach at the optimal solution by taking optimal subsolutions of its sub problems.

Principle optimality in Greedy Approach.

Greedy choose solution, which seems best at the moment.

So In order to find out shortest path from A to J, we start from node A.

From, A → B ⇒ 2

A → C ⇒ 4

A ⇒ D ⇒ 4, edge with smallest value is selected.

i.e., A → B.

From B → E ⇒ 7

B ⇒ F ⇒ 4

B → G ⇒ 6,

B → F path is selected.

From F → A ⇒ 6

F → I ⇒ 3, F → I path is

selected.

So the shortest path obtained in Greedy approach is

$A \rightarrow B \rightarrow F \rightarrow I \rightarrow J$

$2 + 4 + 3 + 4 \Rightarrow 13.$

[which is a near optimal solution]

The main difference between greedy approach and dynamic programming is that,

Greedy method computes its solution by making its choice in a serial forward fashion [predefined steps], never revise previous choices

But Dynamic Programming computes its solution bottom up by synthesizing them from smaller subsolutions and by trying many possibilities and choices before it arrives

at the optimal set of choices.

Comparison B/W Dynamic Programming AND Divide and Conquer

Dynamic Programming, like the divide and conquer method, solves problems by combining the solutions to subproblems.

Divide and Conquer	Dynamic Programming
1⇒ The divide and conquer paradigm involves three steps at each level of the recursion ⇒ Divide the problem into no of sub-problems.	The development of a dynamic programming algorithm can be broken into a sequence of 4 steps. ⇒ Characterize the structure of an optimal solution

	Divide and Conquer	Dynamic Programming
	⇒ Conquer the sub problems by solving them recursively. ⇒ Combine the solutions to the sub problems to the soln for the original problems.	⇒ Recursively define the value of an optimal solution ⇒ Compute the value of an optimal solution in a bottom up fashion ⇒ Construct an optimal solution
2⇒	Subproblems call themselves recursively one or more times to deal with closely related subproblems.	Dynamic programming is not recursive.
3⇒	Divide and Conquer does more work on the subproblems and hence has more time consumption.	Dynamic Programming Solves the subproblems and then stores it in the table.
4⇒	Subproblems are independent of each other.	Subproblems are not independent.

	Divide and Conquer	Dynamic Programming.
5⇒	Only one decision sequence is generated	Many decision sequence may be generated.
6⇒	Can be used for any kind of problems	Generally used for optimization problems
7⇒	Examples: ⇒ Merge Sort ⇒ Binary Search ⇒ Strassen's matrix Multiplication	Examples: ⇒ Matrix chain Multiplication ⇒ Floyd Warshall Algorithm ⇒ CYK Algm.

Matrix Chain Multiplication

Matrix chain multiplication problem is to find optimal parenthesization of a chain of matrices to be multiplied such that the number of scalar multiplication is minimized.

For example consider 3 matrices of $A_{5\times4}$ $B_{4\times3}$ and $C_{3\times5}$ then the possible multiplications

$(AB)C \Rightarrow (5\times4\times3) + (5\times3\times5)$

$\Rightarrow 135$

$A(BC) \Rightarrow (5\times4\times5) + (4\times3\times5)$

$\Rightarrow 160.$

From this we can see that by changing the evaluation

sequence, cost of operations also changes.

To find directly the best possible way to calculate the product, we could simply parenthesize the expression in every possible fashion and count each time how many scalar multiplications are required.

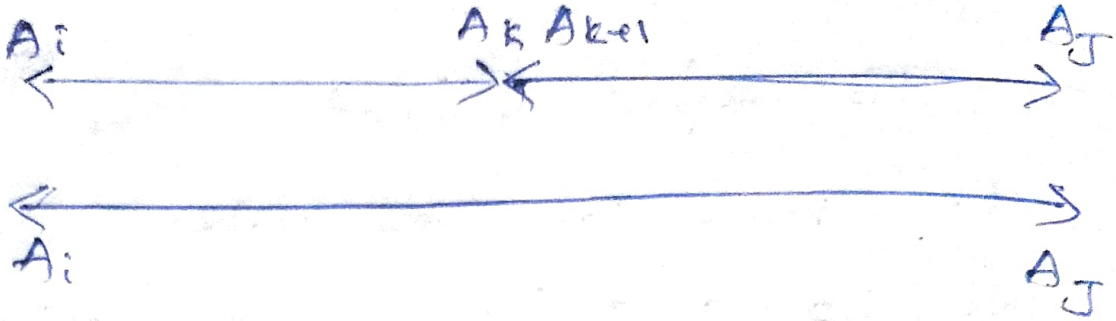
Dynamic Programming Approach for chained matrix multiplication

Step 1:- Characterize the structure of optimal solution.

Let $A_i \ldots A_j$ $(i \le j)$ denote the matrix resulting from $A_i * A_{i+1} \ldots * A_j$.

Any parenthesization of $A_i, A_{i+1} \ldots A_j$ must split the product between A_k and A_{k+1}.

for some k $i \le k < j$.



Step2 :- Recursively define the value of optimal solution

Let m(i,j) be the minimum number of scalar multiplication needed to compute $A_i \times \dots A_j$, where A_i has dimension $P_{i-1} \times P_i$

The m(i,j) can be defined as

$$m(i,j) = \begin{cases} 0 & \text{if } i = j \\ m[i,k] + m[k+1,j] + P_{i-1} P_k P_j & \text{if } i < j \end{cases}$$

Step 3:- Computing the optimal cost

Given a sequence $P = P_0, P_1 \ldots P_n$ $m[i,j]$ records the optimal cost, and $s[i,j]$ records the index k for the corresponding optimal cost.

Algorithm

```
Matrix Chain Order (P)

n ← length [P] - 1
for i ← 1 to n
    m[i, i] ← 0
for l ← 2 to n
    for i ← 1 to n - l + 1
        J ← i + l - 1
        m[i, J] ← ∞
    for k ← i to J - 1
        q ← m[i, k] + m[k+1, J] +
                P_{i-1} P_k P_J
        if q < m[i, J]
            m[i, J] ← q
            s[i, J] ← k
```

Step 4:- Constructing an optimal Solution

Matrix chain order only determines the optimal number of scalar multiplications needed to compute matrix chain product. It doesnot directly show how to multiply matrices.

The procedure Print Optimal (s, i, j) prints the optimal parenthesization $A_i \ldots A_j$.

Algorithm

```
Print Optimal (s, i, j)
if i = j
    Print Ai
else
    Print "("
        Print Optimal (s, i, s[i,j])
        Print Optimal (s, s[i,j]+1, j)
    Print ")"
```

MATRIX CHAIN MULTIPLICATION

Eg:- A[100x10] B[10x5] C[5x50]

P_0 P_1 P_2 P_3

P = {100, 10, 5, 50}

i \ J	1	2	3
1	0	5000	30,000
2		0	2500
3			0

i \ J	1	2	3
1		1	2
2			2
3			

$m[i,i] = 0$ $m[1,1]=0$ $m[2,2]=0$

$m[3,3]=0$

$m[i,j] = m[i,k] + m[k+1,j] + P_{i-1} P_k P_j$

$m[1,2] = m[1,1] + m[2,2] + P_0 P_1 P_2$

$= 0 + 0 + 100 \times 10 \times 5$

$= 5000$

$m[2,3] = m[2,2] + m[3,3] + P_1 P_2 P_3$

$= 0 + 0 + 10 \times 5 \times 50$

$= 2500$

$k=1$ $m[1,3] = m[1,1] + m[2,3] + P_0 P_1 P_3$

$= 0 + 2500 + 100 \times 10 \times 50$

$= 52500$

$k=2$

$$m[1,3] = m[1,2] + m[3,3] + P_0 P_2 P_3$$

$$= 5000 + 0 + 100 \times 5 \times 50$$

$$= 30,000$$

Optimal Parenthesization

Print Optimal Parens (S, 1, 3)

if i = j [1 ≠ 3]

else

"("

Print Optimal Parens (S, i, S[i,j])

Print Optimal Parens (S, S[i,j]+1, j)

")"

⇒ "(" POP (S, 1, S[1,3]) POP (S, S[1,3]+1, 3) ")"

⇒ "(" POP (S,1,2) POP (S, 3,3) ")"

"(" "(" POP (S,1,S[1,2]) (S, S[1,2]+1, 2) ")" ")"

POP (S,3,3) ")"

⇒ "(" "(" POP (S,1,1) POP (S,2,2) ")"

POP (S,3,3) ")"

POP (S,1,1)

i = j

So Print A,

POP (3,2,2)

i = J

So Print A_2.

POP (5, 3, 3)

i = J

So Print A_3

"(" (A_1 A_2) A_3)

FLOYD - WARSHALL ALGORITHM

Floyd-Warshall algorithm [Also known as the Roy-Floyd Algorithm] is used to solve all pairs shortest path problem on a directed weighted graph.

It runs in $\Theta(V^3)$ time.

This algorithm is based upon dynamic programming technique, in which the key part is to reduce a large problem into smaller problems.

This can be done by limiting the set of vertices through which the path is allowed to pass.

Step 1: Structure of the optimal solution
[Structure of Shortest Path]

Let the vertices of graph G are $V = \{1, 2, \ldots n\}$.

Let us consider a subset $\{1, 2 \ldots k\}$ of vertices for some k.

For any pair of vertices $i, j \in V$ consider all paths from i to j. whose intermediate vertices are all drawn from $\{1, 2 \ldots k\}$ and Let p be a minimum Weight path from among them.

The relationship between path p and shortest paths from i to j depends on whether or not k is an intermediate vertex of path p.

62

⇒ If k is not an intermediate vertex of path p, then all intermediate vertices of path p are in the set $\{1, 2 \ldots k-1\}$.

⇒ If k is an intermediate vertex of path p, then we break down p into $i \xrightarrow{P_1} k$ and $k+1 \xrightarrow{P_2} j$.

Step 2 :- A Recursive Solution to the all pairs shortest path problem.

Let $d_{ij}^{(k)}$ be the weight of a shortest path from vertex i to vertex j for which all intermediate vertices are in the set $\{1, 2 \ldots k\}$.

$$d_{ij}^{(k)} = \begin{cases} w_{ij} & \text{if } k = 0 \\ \min\left(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\right) & \text{if } k \geq 1 \end{cases}$$

Step 3 :- Computing the shortest path

The procedure Floyd-Warshall(W) will produce the matrix $D^{(n)}$ of shortest path weights.

Its input is an $n \times n$ matrix W i.e adjacency matrix of graph G.

Algorithm

```
Floyd-Marshall(W)
n ← rows[W]
D^(0) ← W
for k ← 1 to n
    do for i ← 1 to n
        for j ← 1 to n.
        do. d_ij^(k) ← min(d_ij^(k-1), d_ik^(k-1) + d_kj^(k-1))
return D^(n).
```

Step 4 :- Constructing Shortest path

From the matrix D of shortest path weights [which is calculated in step 3] we can construct the predecessor matrix Π from the D metrix.

Given the predecessor matrix Π the ~~print~~ Print_All_Pairs_Shortest_Path ~~Pro~~ procedure can be used to print the vertices on a given shortest path.

The predecessor matrix Π can be calculated from the equations given below:-

When $k = 0$

$$\pi_{ij}^{(0)} = \begin{cases} NIL & \text{if } i = j \text{ or } w_{ij} = \infty \\ i & \text{if } i \neq j \text{ \& } w_{ij} < \infty \end{cases}$$

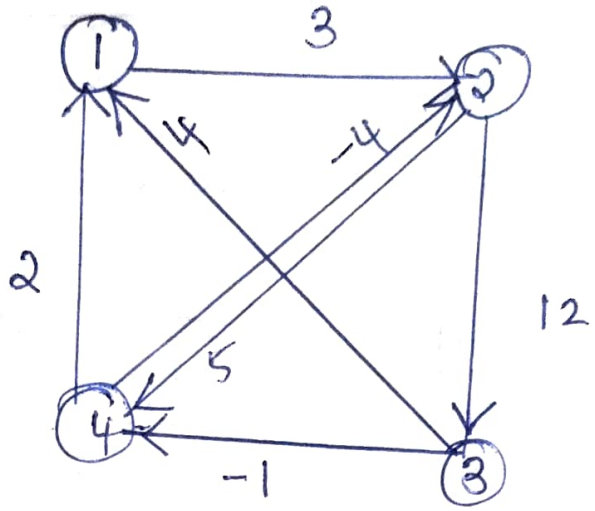
For $k \geq 1$

$$\pi_{ij}^{(k)} = \begin{cases} \pi_{ij}^{(k-1)} & \text{if } d_{ij}^{(k-1)} \leq d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \\ \pi_{kj}^{(k-1)} & \text{if } d_{ij}^{(k-1)} > d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \end{cases}$$

Algorithm

```
Print_All_Pairs_Shortest_Path (Π, i, j)
if i = j
   then print i
   else if π_ij = NIL
      then print "no path from" i
           'to' j "exists"
   else
      Print_All_Pairs_Shortest_Path (Π, i, π_kj)
      print j.
```

Q). Find the shortest path from the given graph using Floyd-Warshall's algorithm.



$$D^0 = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 3 & \infty & \infty \\ \infty & 0 & 12 & 5 \\ 4 & \infty & 0 & -1 \\ 2 & -4 & \infty & 0 \end{bmatrix} \end{array}$$

It is constructed as follows:

$$d_{ij}^0 = \begin{cases} 0 & \text{if } i = j \\ w & \text{if } i \neq j, (i,j) \in E \\ \infty & \text{if } i \neq j \text{ \& } (i,j) \notin E \end{cases}$$

$$\pi_{ij}^{0} = \begin{cases} Nil & \text{if } i = j \text{ or } w_{ij} = \infty \\ i & \text{if } i \neq j \text{ or } w_{ij} < \infty \end{cases}$$

$$\pi^{0} = \begin{bmatrix} Nil & 1 & Nil & Nil \\ Nil & Nil & 2 & 2 \\ 3 & Nil & Nil & 3 \\ 4 & 4 & Nil & Nil \end{bmatrix}$$

$$D^{1} = \begin{bmatrix} 0 & 3 & \infty & \infty \\ \infty & 0 & 12 & 5 \\ 4 & (7) & 0 & -1 \\ 2 & -4 & \infty & 0 \end{bmatrix} \qquad \pi^{1} = \begin{bmatrix} Nil & 1 & Nil & Nil \\ Nil & Nil & 2 & 2 \\ 3 & 1 & Nil & 3 \\ 4 & 4 & Nil & Nil \end{bmatrix}$$

$d_{23}, d_{21} + d_{13}$

(12), $\infty + \infty = \infty$

$d_{24}, d_{21} + d_{14}$

(5), $\infty + \infty = \infty$

$d_{32}, d_{31} + d_{12}$

∞, $4 + 3 = (7)$

$d_{34}, d_{31} + d_{14}$

(-1) $4 + \infty = \infty$

$d_{42}, d_{41} + d_{12}$

(4), $2 + 3 = 7$

$d_{43}, d_{41} + d_{13}$

(∞), $2 + \infty = \infty$

Keep the first row and first column & diagonal elts same as D^{0}.

$$D^2 = \begin{bmatrix} 0 & 3 & (5) & (8) \\ \infty & 0 & 12 & 5 \\ 4 & 7 & 0 & -1 \\ 2 & -4 & (8) & 0 \end{bmatrix} \qquad \Pi^2 = \begin{bmatrix} Nil & 1 & 2 & 2 \\ Nil & Nil & 2 & 2 \\ 3 & 1 & Nil & 3 \\ 4 & 4 & 2 & Nil \end{bmatrix}$$

$d_{31}, d_{32} + d_{21}$

(4), $7 + \infty = \infty$

$d_{34}, d_{32} + d_{24}$

(-1), $7 + 5 = 12$

$d_{41}, d_{42} + d_{21}$

(2), $-4 + \infty = \infty$

$d_{43}, d_{42} + d_{23}$

∞, $-4 + 12 = (8)$

$$D^3 = \begin{bmatrix} 0 & 3 & 15 & 8 \\ (16) & 0 & 12 & 5 \\ 4 & 7 & 0 & -1 \\ 2 & -4 & 8 & 0 \end{bmatrix} \qquad \Pi^3 \begin{bmatrix} Nil & 1 & 2 & 2 \\ 3 & Nil & 2 & 2 \\ 3 & 1 & Nil & 3 \\ 4 & 4 & 2 & Nil \end{bmatrix}$$

$d_{12}, d_{13} + d_{23}$

(8), $15 + 12 \Rightarrow 27$

$d_{21}, d_{23} + d_{31}$

∞ $12 + 4 \Rightarrow (16)$

$d_{24}, d_{23} + d_{34}$

(5), $12 + -1 \Rightarrow 11$

$d_{41}, d_{43} + d_{31}$

(2), $8 + 4 \Rightarrow 12$

$d_{42}, d_{43} + d_{32}$

(-4), $8 + 7 \Rightarrow 15$

$$D^4 = \begin{bmatrix} 0 & 3 & 15 & 8 \\ 7 & 0 & 12 & 5 \\ 1 & -5 & 0 & -1 \\ 2 & -4 & 8 & 0 \end{bmatrix} \qquad \Pi^4 = \begin{bmatrix} Nil & 1 & 2 & 2 \\ 4 & Nil & 2 & 2 \\ 4 & 4 & Nil & 3 \\ 4 & 4 & 2 & Nil \end{bmatrix}$$

$d_{12}, d_{14} + d_{42}$
3, $8 + -4 = 4$

$d_{13}, d_{14} + d_{43}$
15, $8 + 8 = 16$

$d_{21}, d_{24} + d_{41}$
16, $5 + 2 \Rightarrow 7$

$d_{23}, d_{24} + d_{43}$
12, $5 + 8 \Rightarrow 13$

$d_{31}, d_{34} + d_{41}$
4, $-1 + 2 \Rightarrow 1$

$d_{32}, d_{34} + d_{42}$
7, $-1 + -4 \Rightarrow -5$

In order to find the shortest path between $1 \rightarrow 4$.

$\Rightarrow$ check the entry d_{14}^5

$$d_{14}^5 = 8.$$

$\Rightarrow$ So shortest distance between $1 \rightarrow 4$ is 8.

$\Rightarrow$ We can find out the corresponding path using predecessor matrix and Print_All_Pair_Shortest_Path Procedure.

⇒ It calls $(\Pi, i, j) \Rightarrow (\Pi, 1, 4)$

check if $i = j$ ie $1 = 4$

⇒ No.

else if check $\Pi^{5}_{14} \Rightarrow$ Nil

$\Pi^{5}_{14} = 2 \neq$ Nil

else

Print APSP (Π, i, Π_{ij})

$\Rightarrow (\Pi, 1, 2)$

print $j \Rightarrow$ Print 4

4

$\Rightarrow (\Pi, 1, 2)$

check, $i = j$ $1 \neq 2$.

else if $\Pi_{12} =$ Nil

$\Pi_{12} = 1 \neq$ Nil

else

Print APSP (Π, i, Π_{ij})

$\Rightarrow (\Pi, 1, 1)$

print $j \Rightarrow$ Print 2

2	4

⇒ (TT, 1, 1)

check i = j 1 = 1 ⇒ true

then print i ⇒ print 1

1	2	4

⇒ Shortest path between 1 → 4

is 1 → 2 → 4 with distance 8.

Back Tracking

Many problems which deal with searching for a set of solutions or which ask for an optimal solution satisfying some constraints can be solved using backward formulation.

Backtracking is a methodical way of trying out various sequence of decisions until we find out one that works.

The key point for the backtracking algorithm is binary choice that means "Yes" or "No".

Whenever the backtracking have choice "No" that means the algorithm has encountered a dead end and it backtracks one step and tries a different path for choice "Yes".

That is, backtracking proceeds as follows:

1. Generate solution tree with all possible cases
2. Test each solution step by step.
3. Once the path is recognized as dead end
   ⇒ Mark the path as infeasible
   ⇒ Go back one step.
   ⇒ Continue with step 2.

Control Abstraction of Backtracking.

```
Backtrack (n)
k ← 1
While k > 0, do
    if there remained an untried
        x(k) such that x[k] ∈ T[x[1]
        ... x[k-1]] and
        Bk[x[1] ... x[k]] = True
```

```
then,
    if [x[1] ... x[k] is a path
       to an answer node, then.
          print [x[1] ... x[k]]
    end if
k ← k+1
    else
       k ← k-1
    end if
end Backtrack.
```

The N Queen's Problem

This problem is to place n-queens in an n-by-n chess board such that no two queens attack each other by being in the same row, column or diagonal.

The solution space consists of all n! permutations of the n-tuple (1, 2 ... n).

8 Queen's Problem

8 Queen problem is to place 8 queens in an 8 by 8 chessboard by satisfying the following conditions.

⇒ Each row should contain atmost one queen.

⇒ No two queens should come

in same row, column and diagonal to each other.

Similarly 4 Queens problem is to place 4 Queens in a 4 x 4 chess board.

Solution to 4 Queen's Problem.

⇒ Place first queen Q_1 in the first column.

(1)

Q_1	x	x	x
x	x		
x		x	
x			x

⇒ Place the second queen Q_2 in the third column of 2nd row.

(2)

Q_1	x	x	x
x	x	Q_2	x
x	x	x	x
x		x	x

⇒ deadend.

⇒ After placing 1st and 2nd queen, we cannot place 3rd queen anywhere.

⇒ So we backtrap one step. (in to 1).

⇒ Place Q_2 in 4th column.

(3)

Q_1	x	x	x
x	x	x	Q_2
x		x	x
x	x		x

⇒ Place Q_3 in 2nd column.

(4)

Q_1	x	x	x
x	x	x	Q_2
x	Q_3	x	x
x	x	x	x

deadend

⇒ Here we have no option to place Q_4.

⇒ So we back track [in to 1]

[There are no other options available in 3].

⇒ By backtracking we place the 1st queen in 2nd column.

X	Q_1	X	X
X	X	X	
	X		X
	X		

⇒ Place 2nd queen in 4th column.

X	Q_1	X	X
X	X	X	Q_2
	X	X	X
	X		X

⇒ Place 3rd Queen in 1st column.

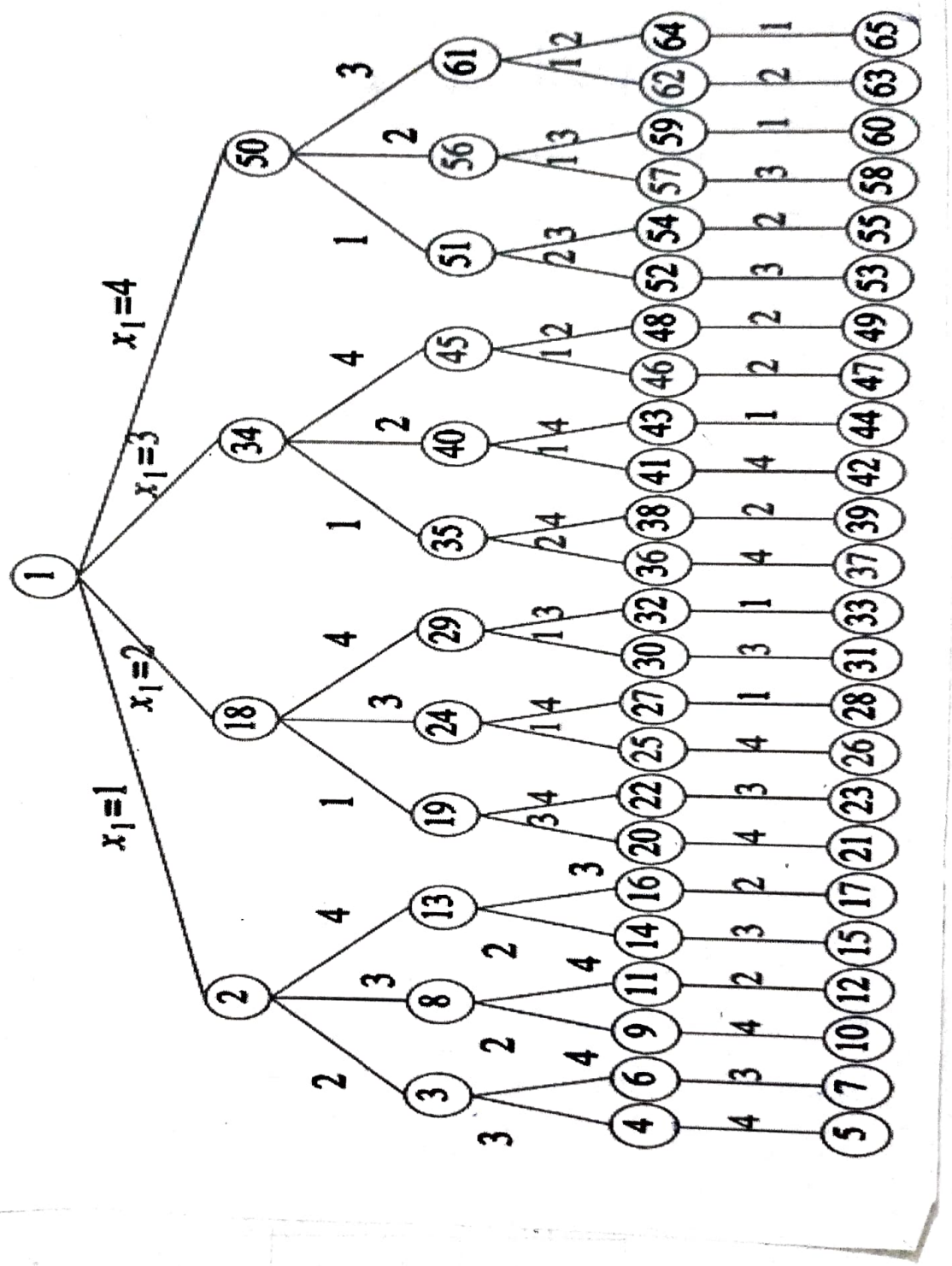
X	Q_1	X	X
X	X	X	Q_2
Q_3	X	X	X
X	X		X

⇒ Place 4th Queen in 3rd column.

X	Q_1	X	X
X	X	X	Q_2
Q_3	X	X	X
X	X	Q_4	X

Solution

Thus the solution to 4 Queens problem is (2, 4, 1, 3).



Backtracking algorithms determines problem solutions by systematically searching the solution space for the given problem instance.

This search is facilitated by using a tree organization for the solution space.

Each node in the tree defines a "problem state".

All paths from the root to other nodes define the "state space" of the problem.

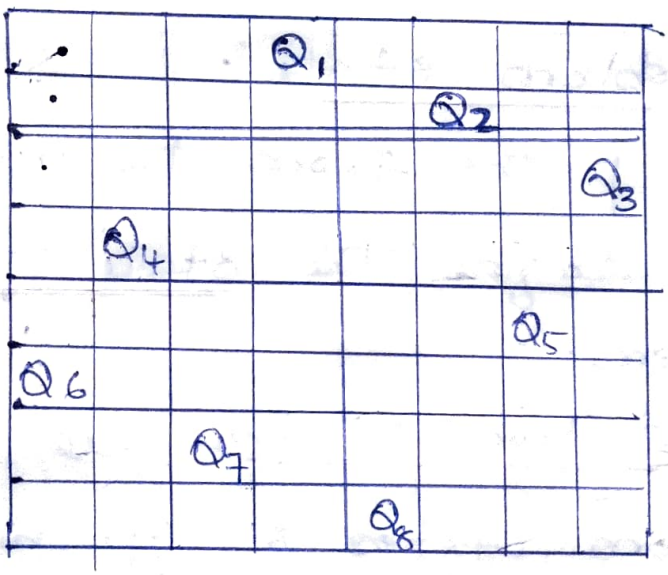
"Solution states" are those problem states 's' for which the path from the root to 's' defines a tuple in the solution space.

"Answer states" are those solutions states 's' for which the path from the root to 's' defines a tuple that is a member of the set of

solutions [i.e., it satisfies the implicit constraints] of the problem.

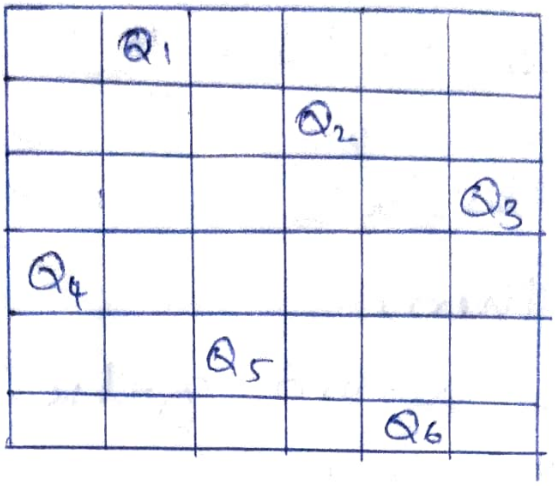
The tree organization of the solution space is referred to as the "state space tree".

Solution to 8 Queen's Problem



Solution ⇒ {4, 6, 8, 2, 7, 1, 3, 5}

Solution to 6 Queen's Problem.



Solution ⇒ { 2, 4, 6, 1, 3, 5 }

Algorithm

```
NQueens(k, n)
for i ← 1 to n
    if (Place(k, i), then.
        x[k] ← i
        if (k == n), then.
            Write(x[1..n]);
        else
            NQueens(k+1, n);
```

⇒ Generate all solutions.

```
Place(k, i)
    for J ← 1 to k-1
        if x[J] = i
            or
        abs(x[J]-i) = abs(J-k)
        then,
            return false.
    return true.
```

⇒ This algorithm check whether new queen can be placed or not.

if "x[J] = i" ⇒ Queens in the same column.

if "abs(x[J]-i) = abs(J-k)" ⇒ Queens are in same diagonal.

If these two conditions are false we can place new queen to that position.

Branch and Bound.

Branch and bound is a systematic method for solving optimization problems.

This method is applied where the greedy method and dynamic programming fails.

However it is much slower, it often leads to exponential time complexities in the worst case.

Branch and bound procedures requires two tools.

⇒ Branching, and

⇒ Bounding.

"Branching" is a way of covering the feasible region by several smaller feasible sub regions.

"Bounding" is a fast way of finding upper and lower bounds

for the optimal solution within a feasible subregion.

Like backtracking, branch and bound technique explores the implicit graph and deals with the optimal solution to given problem.

In this technique at each stage we calculate the bound for a particular node and check whether this bound will be able to give the solution or not.

If we find that at any node the solution so obtained is appropriate but the remaining solution is not leading to a best case then we leave this node without exploring.

At each stage we have for

each node a bound - lower bound for minimization problems and upper bound for maximization problems.

The calculated bound is checked with previous best result and if found that new partial solution results leads to worse case, than bound with the best solution, we leave this part without exploring it further.

* The core of this approach is a simple observation that if the lower bound for a sub region A from the search tree is greater than the upper bound for any other sub region B, then A may be safely discarded from the search.

This step is called "Pruning".

The efficiency of the method depends critically on the effectiveness

of the branching and bounding algorithms used.

Bad choices could lead to repeated branching without any pruning, until the subregions becomes very small.

Travelling Salesman Problem.

TSP is a problem in which a salesperson has to visit a number of cities during a tour and the condition is to visit all the cities exactly once and return back to the same city where the person started.

Let $G = (V, E)$ be a directed graph defining an instance of TSP.

Let $c(i, j)$ be the cost matrix.

$$c_{ij} = \begin{cases} \text{the cost of the edge} & \text{; if there is a path b/w vertex 'i' and vertex 'j'} \\ \infty & \text{; if there is no path.} \end{cases}$$

Algorithm

Let $G = (V, E)$ be a directed graph defining an instance of the TSP.

1: Graph is represented by a cost matrix

2: Convert cost matrix into reduced matrix A.
[i.e., every row and column should contain atleast one zero entry].

3: Cost of the reduced matrix $\hat{c}(R)$, is the sum of elements that are subtracted from rows and columns of cost matrix to make it reduced.

4: Make the state space tree for reduced matrix.

5: for each (i, j) node to be included

a:- change all entries of row-i and column-j of A to infinity.

b: Set A(j, 1) to infinity.

c: Reduce all rows and columns in the resulting matrix except for rows and columns containing only infinity.

6: Calculate the cost of the matrix.

$$\hat{c}(s) = \hat{c}(R) + A(i,j) + r$$

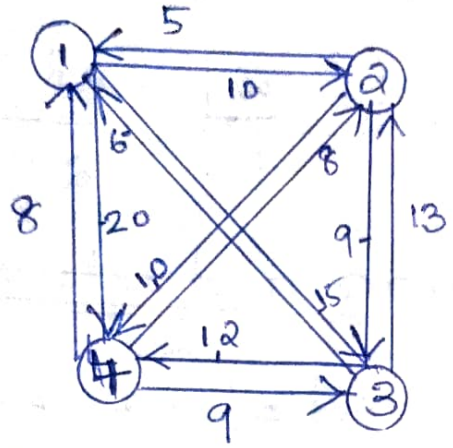
where A(i,j) is the cost of the edge (i,j) in the reduced matrix.

$\hat{c}(R)$ ⇒ Cost of the original reduced cost matrix, and

r ⇒ new reduced cost matrix.

7: Repeat the above steps for all the nodes until all the nodes are generated and we get a path.

Q) Apply branch and bound technique to solve TSP for the graph given



Solution

(1) Cost matrix

	1	2	3	4
1	∞	10	15	20
2	5	∞	9	10
3	6	13	∞	12
4	8	8	9	∞

(2) Reduced matrix

Perform Row reduction

Find the smallest value in the row and subtract it from all other values.

$$\begin{bmatrix} \infty & 0 & 5 & 10 \\ 0 & \infty & 4 & 5 \\ 0 & 7 & \infty & 6 \\ 0 & 0 & 1 & \infty \end{bmatrix} \quad \begin{matrix} 10 \\ 5 \\ 6 \\ 8 \end{matrix} \quad \begin{matrix} + \\ \\ \\ \\ \hline 29 \end{matrix}$$

Perform Column reduction

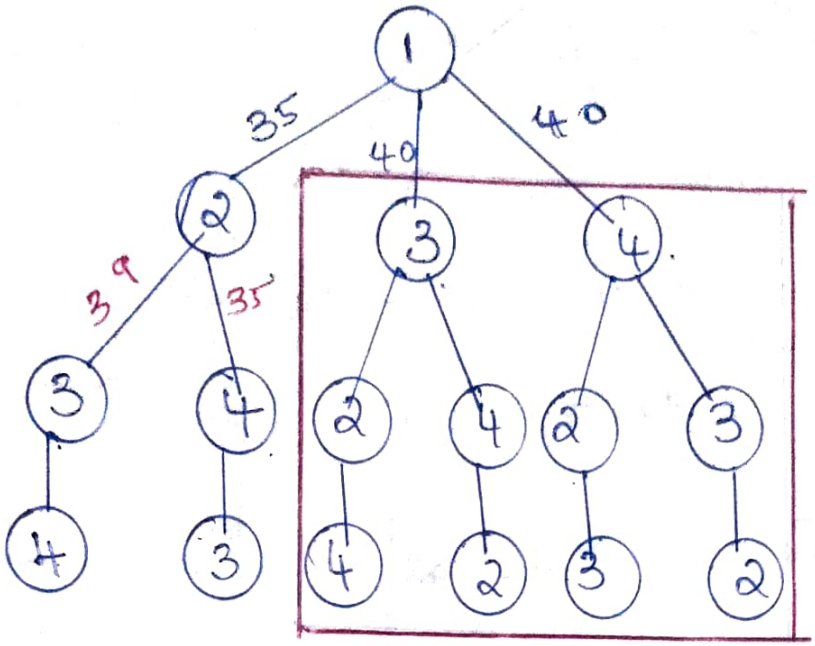
Find the smallest element in the column, subtract it from all other entries.

$$A = \begin{bmatrix} \infty & 0 & 4 & 5 \\ 0 & \infty & 3 & 0 \\ 0 & 7 & \infty & 1 \\ 0 & 0 & 0 & \infty \end{bmatrix}$$

$1 + 5 \Rightarrow 6$

(3) $\hat{c}(8) \Rightarrow 10 + 5 + 6 + 8 + 1 + 5 \Rightarrow 35$

(4) Construct state space tree.



(5) (1 – 2) node

⇒ PRUNING

a, b:- 1st row
2nd column
(2, 1) entry
} ∞

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ \infty & \infty & 3 & 0 \\ 0 & \infty & \infty & 1 \\ 0 & \infty & 0 & \infty \end{bmatrix}$$

c: Check whether it is reduced or not.

⇒ It is reduced.

$$\hat{c}(s) = \hat{c}(R) + A(i,j) + r$$

$$= 35 + 0 + 0 = 35$$

(1 - 3) node

a, b:-

1st row
3rd colum
(3, 1) entry
} ∞

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & 0 \\ \infty & 7 & \infty & 1 \\ 0 & 0 & \infty & \infty \end{bmatrix}$$

c. check whether it is reduced or not.

⇒ It is not reduced

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty \\ \infty & 6 & \infty & 0 \\ 0 & 0 & \infty & \infty \end{bmatrix} \quad 1$$

$$\hat{c}(S) = \hat{c}(R) + A(i,j) + r$$

$$= 35 + 4 + 1 \Rightarrow 40$$

(1-4) node

a,b

1st row

4th column.

(4,1) entry

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ 0 & \infty & 3 & \infty \\ 0 & 7 & \infty & \infty \\ \infty & 0 & 0 & \infty \end{bmatrix}$$

c

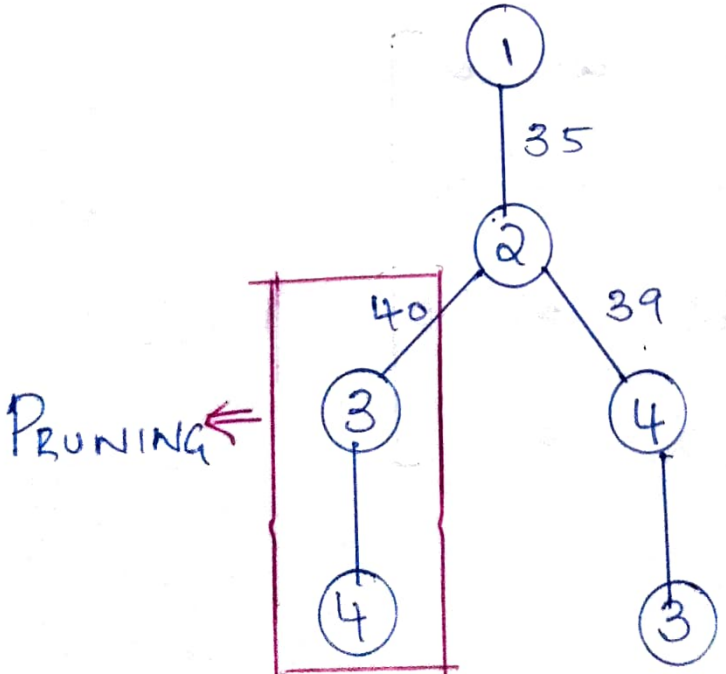
Check whether it is reduced?

⇒ It is reduced

$\hat{c}(s) = \hat{c}(R) + A(i,j) + r$

$= 35 + 5 + 0 = 40$

So the path 1-2 is selected.



(2-3) model

a, b:-

2nd row

3rd column } ∝

(3,1) entry

NB:- Take (1-2) reduced matrix

$$\begin{bmatrix} \propto & \propto & \propto & \propto \\ \propto & \propto & \propto & \propto \\ \propto & \propto & \propto & 1 \\ 0 & \propto & \propto & 0 \end{bmatrix}$$

c

Check whether it is reduced or not

⇒ It is not reduced.

$$\begin{bmatrix} \propto & \propto & \propto & \propto \\ \propto & \propto & \propto & \propto \\ \propto & \propto & \propto & 0 \\ 0 & \propto & \propto & 0 \end{bmatrix} \quad 1$$

$\hat{c}(S) = \hat{c}(R) + A(i,j) + r$

$= 35 + 3 + 1 \Rightarrow 39$

(2-4) node

a, b :-

2nd row
4th column
(4,1) entry
} ∞

NB: Take (1-2) reduced matrix.

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty \\ \infty & \infty & 0 & \infty \end{bmatrix}$$

c

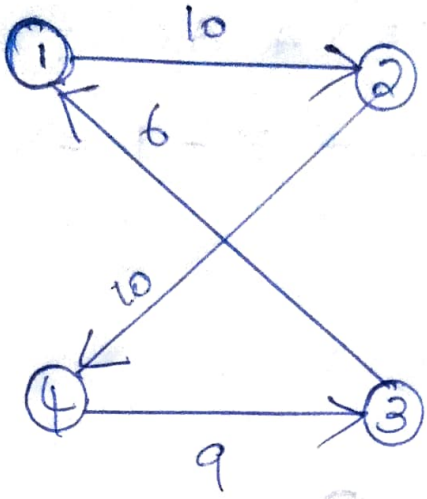
Check whether it is reduced or not

$\Rightarrow$ It is reduced

$\hat{c}(S) = \hat{c}(R) + A(i,j) + r$

$= 35 + 0 + 0 \Rightarrow 35$

So the path is $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$



Cost ⇒ 10 + 10 + 9 + 6 ⇒ 35.